Exception Handling and Post-Release Defects: A Replication with Predictive and Explainable Extensions

Milestone 3: Preliminary Findings

Dev Patel *101042729* | Priyanka Gupta *101008820* | Dhruv Patel *101037734* | Slesha Vora *101043167* | Pulkit Chopra *101030196*

Ontario Tech University, Oshawa, ON, Canada

Software Maintenance and Evolution · Group 8 · Project Type: Replication

Artifacts: github.com/DevPatel1704/exception-handling-defects-replication (git tag milestone-3)

***Abstract*—This checkpoint reports the evidence produced by our replication of de Pádua and Shang's MSR 2018 study of exception handling (EH) and post-release defects. Reproduction: with a one-line fix to the released scripts, all nine optimism-reduced R^2 values of the paper's Table 4 reproduce within ±0.004. New context: on four newly collected Java releases (Hibernate ORM 5.1.0; Apache ZooKeeper 3.6.0, 3.7.0, 3.9.0), Hibernate keeps the paper's ordering (BASE 0.238 < BSAP 0.264 < BSFC 0.275). On ZooKeeper, EH models first appear worse than the baseline; a sensitivity analysis shows this is an artefact of the released model-construction code, and with the paper's stated rule EH models match or slightly exceed the baseline (−0.006 to +0.035). Extensions: across all seven contexts, adding EH metrics to logistic regression, random forest or XGBoost yields no prediction gain that survives Holm correction (0 of 189 tests). Validity: over 10 analyzer runs per release, anti-pattern metrics never change, but six flow metrics do, and they can move one release's BSFC result from R^2 0.04 to 0.46.**

***Index Terms*—Exception handling, post-release defects, replication, defect prediction, SHAP, measurement validity.**

I. Research Questions and Progress

We replicate de Pádua and Shang [1], who modelled whether exception-flow characteristics and catch anti-patterns explain post-release defects beyond traditional metrics, and extend the study with prediction and explanation. **Scope:** Java releases (plus the paper's C# Umbraco for reproduction only); unit of analysis is the source file; as in the paper's presented models, only files with at least one catch block; outcome = at least one post-release defect (a fixed Jira bug reported after the release and linked to the file through its fixing commit). All models are associational. Our current questions are:

RQ1 (replication) Do exception-flow metrics (BSFC model) add explanatory power over traditional product and process metrics (BASE)?

RQ2 (replication) Do the 17 catch anti-patterns (BSAP model) add explanatory power over BASE?

RQ3 (extension) Do EH metrics improve machine-learning prediction (PR-AUC) of post-release-defective files?

RQ4 (extension) Which EH features drive those predictions (SHAP [13]), and do they agree with the significant predictors of RQ1–RQ2?

Table I links every question to the evidence that answers it. The questions are unchanged since Milestone 2; the scope and methods changed as listed in Table II, each with its reason.

TABLE I
RESEARCH QUESTIONS AND THE EVIDENCE THAT ANSWERS THEM

Question	Evidence in this report	Status
RQ1, RQ2: reproduction	Table IV (upper), Obs. 1	Answered
RQ1, RQ2: new context	Tables IV (lower), V; Fig. 1; Obs. 2–3	Answered, 4 releases
RQ3	Fig. 2, Table VI, Obs. 4	Answered, 7 contexts
RQ4	Table VI, Obs. 5	Answered, 7 contexts
Validity of RQ1 (new data)	Fig. 3, Sec. IV-B (check V1)	Run on 3 releases
Validity of labels	Sec. IV-C (check V2)	Prepared, rating pending

TABLE II
CHANGES SINCE MILESTONE 2 AND THEIR JUSTIFICATION

Change	Why
RQ3/RQ4: protocol → results	Protocol was fixed in Milestone 2; now run on all seven contexts
+ ZooKeeper 3.7.0 and 3.9.0	Three consecutive lines of one system test stability over time; replaces Hadoop 2.7
− Hadoop 2.7 (dropped)	JTratch already needs a 6 GB heap for the smaller Hibernate; our machine has 7.4 GB
− C# new context (dropped)	Umbraco 7's YouTrack tracker is decommissioned: no defect labels
− Javadoc DB, dependency jars (dropped)	Rebuilding the authors' unpublished Javadoc DB is out of scope; flow-metric ranks are preserved ($\rho \geq 0.63$), kept as a threat
+ Holm correction for RQ3	27 tests per context inflate false positives; it overturns our earlier Umbraco ROC-AUC claim
New-context ML: 5×5 → 5×10 CV	Same protocol as on the authors' data
+ Sensitivity S1, guard D7	Explain Obs. 6 of Milestone 2; released code halts on an empty model
Raters: Priyanka → Dev, Dhruv	Raters independent of the defect-mining code author

II. Methods Executed

A. Data

Table III lists the seven study contexts. The three original contexts use the authors' released file-level dataset [16] unchanged. The four new contexts were collected by our pipeline from the tagged release commits (full SHAs in `docs/DATA.md`); Jira issues were retrieved through the anonymous REST API on 2026-10-24 (ZooKeeper 3.7.0: 2026-11-08) and the responses are committed, so labels can be recomputed offline.

TABLE III
STUDY CONTEXTS (MODELLED FILES = ≥ 1 CATCH BLOCK WITH TRY DATA)

Context	Release → post-release	Commit	Modelled (defective)
Original study: authors' released data [16]			
Umbraco (C#)	7.6.0 → 7.6.12	c1853e8	230 (22)
Hadoop	2.6.0 → 2.6.5	e349649	890 (143)
Hibernate ORM	5.0.0 → 5.0.16	87a6226	440 (93)
New context: collected by us			
Hibernate ORM	5.1.0 → 5.1.17	a909e76	404 (86)
ZooKeeper	3.6.0 → 3.6.4	b4c89dc	161 (30)
ZooKeeper	3.7.0 → 3.7.2	e3704b3	169 (17)
ZooKeeper	3.9.0 → 3.9.6	1674a5e	178 (39)

Release files: 3,475 / 469 / 487 / 504 for the new contexts; post-release defect issues retained: 182 / 43 / 16 / 35. Source: `data/new/<id>/dataset_manifest.json`, `defect_mining_stats.json`.

B. Procedures

(1) Explanatory models (RQ1–RQ2). We run the authors' two R notebooks unchanged except for three recorded patches: D1 fixes the bug that fits BSAP's data into BSFC; D2 makes an R ≥ 4.3 `&&` semantics explicit; D7 (new) initialises `refit` so that a model whose backward elimination keeps no predictor is recorded as empty instead of halting the run. Models are logistic (`rms::lrm`); predictors are reduced by Spearman correlation ($|\rho| > 0.7$), redundancy analysis and an n/15 degrees-of-freedom budget, simplified by `fastbw`, and evaluated by optimism-reduced Nagelkerke R^2 (1,000 bootstrap samples, seed 2018).

(2) New-context data. Files under `src/main/java` at the release commit; process metrics over the previous-release..release window; post-release defects as above; 12 open-source product metrics (lizard) because SciTools Understand is unavailable; EH metrics from JTratch (commit af7c232, Java 8), aggregated to file level by our re-implementation of the authors' rule sheet, validated in Milestone 2 (57/107 metrics ≥ 95% identical on Hibernate 5.0).

(3) Prediction (RQ3). Logistic regression (log10 transform, standardised, L2, class-balanced), random forest (500 trees, min. leaf 2) and XGBoost (300 trees, depth 4, learning rate 0.05), all untuned so that only the feature set varies: TRAD vs. TRAD+FLOW, TRAD+AP and TRAD+EH. Repeated stratified 5-fold CV × 10 (50 identical folds per comparison, seed 2018); PR-AUC is primary, ROC-AUC and F1 secondary. Uplift over TRAD is tested with the Nadeau–Bengio corrected resampled t-test [21], and p-values are Holm-adjusted [22] over the 9 comparisons of each context and metric.

(4) Explanation (RQ4). TreeSHAP on the best tree learner (TRAD+EH), refit on all files; a top-10 EH feature *matches* RQ1–RQ2 if it is, or correlates at $|\rho| > 0.7$ with, a significant BSFC/BSAP predictor.

(5) Validity check V1 (analyzer variance). For each ZooKeeper release we ran JTratch 10 more times on the identical checkout, rebuilt the inputs with every other source held fixed, refit all models, and compared every metric and model across runs. **Sensitivity S1:** the paper (§3.4.6) states that BASE-significant metrics are kept in BSFC/BSAP, but the released code guards that branch with a never-true condition. S1 enables it, reserving one budget slot per kept metric. **Check V2 (labels, prepared):** a seeded random sample of 131 issue–commit–file links (40/40/16/35) with a rating packet, a written protocol, two independent sheets, and a script for Cohen's κ [23] and precision with a Wilson interval [24].

C. Tools, settings and artifacts

Windows 11 (12 cores, 7.4 GB RAM); R 4.3.3 (rms 6.8.1, Hmisc 5.2.3, caret 6.0.94); Python 3.11.17 (scikit-learn 1.9.1, XGBoost 3.4.2, SHAP 0.51.0, pandas 3.1.0); OpenJDK 1.8.0_472; conda environment pinned in `environment.yml`. Every result file names the command that regenerates it; `docs/REPRODUCTION.md` §9 lists the Milestone 3 commands, and 13 unit tests check the new statistics against reference values. **LLM use:** no language model is part of the pipeline, so no model identifier, sampling setting or prompt affects any result. Separately, an AI coding assistant (Claude Code, `claude-opus-5-5`, Anthropic; Oct. 24–Nov. 8, 2026) helped draft scripts, documentation and this report's first draft; the team reviewed all code, and every number comes from executing the committed code (disclosure in `CONTRIBUTIONS.md`).

III. PRELIMINARY FINDINGS

We separate reproduction of the original study (Sec. III-A, authors' data) from findings in the new context (Sec. III-B to III-D, our data). Table IV and Fig. 1 give the explanatory models for all seven contexts.

TABLE IV
OPTIMISM-REDUCED NAGELKERKE R^2 OF THE SIMPLIFIED MODELS

Context	BASE	BSFC	BSAP	BSFC (S1)	BSAP (S1)
A. Reproduction, authors' data: ours (paper's Table 4)					
Umbraco 7.6	0.090 (0.09)	0.089 (0.09)	0.176 (0.18)	—	—
Hadoop 2.6	0.331 (0.33)	0.372 (0.37)	0.351 (0.35)	—	—
Hibernate 5.0	0.211 (0.21)	0.279 (0.28)	0.233 (0.23)	—	—
B. New context, our data (as released → S1 sensitivity)					
Hibernate 5.1	0.238	0.275	0.264	0.275	0.264
ZooKeeper 3.6	0.423	0.041[a]	0.417	0.458	0.417
ZooKeeper 3.7	0.417	empty[b]	0.047	0.413	0.413
ZooKeeper 3.9	0.233	0.093	0.157	0.250	0.231

[a] 0.041–0.458 over 10 analyzer runs (Fig. 3). [b] No predictor survives backward elimination (guard D7). S1 = BASE-significant metrics kept as the paper states (§3.4.6). Source: `results/contexts_summary.csv`, `results/new/<id>/fixed{,_forcesig}/`.

A. Reproduction of the original study

Observation 1. As released, BSFC is a copy of BSAP; with the one-line fix (D1), all nine R^2 values match the paper within ±0.004 (Table IV-A), every model-construction indicator matches, and all 35 entries of the paper's Table 5 match with $|\Delta\chi^2| \le 0.005$. The new guard D7 changes nothing: rerunning the full reproduction and the three earlier new-context runs with D7 gives identical predictors and R^2 (max. difference 0).

B. New context: explanatory models (RQ1, RQ2)

Observation 2 (Hibernate one release later). Hibernate 5.1.0 keeps the paper's ordering BASE < BSAP < BSFC, all within 0.035 of the paper's Hibernate 5.0 values; EH metrics add +0.037 (BSFC) and +0.026 (BSAP) R^2, compared with +0.07 and +0.02 in the paper. The base predictors that matter recur (comment lines +, comment-to-code ratio −), but the specific EH predictors do not (Table V): none of the five significant EH predictors of 5.0 is significant in 5.1, which instead selects exception propagation and throwing new exceptions.

Observation 3 (ZooKeeper). As released, BSFC and BSAP fall far *below* BASE (e.g., 3.6: 0.041 vs. 0.423), which cannot reflect weaker information: BSFC and BSAP can contain the BASE metrics. The released reduction step dropped the BASE-significant `# Bugs (pre)` or comment metrics (1, 2 and 3 times in the three logs; never on the paper's data or Hibernate). With

S1, EH models are within −0.006 to +0.035 of BASE, and only two EH predictors remain significant: average exception-type prevalence (3.6, −, as in Hibernate 5.0) and `% Handled with Return` (3.9, +). No anti-pattern is significant in any ZooKeeper BSAP model under S1.

TABLE V
SIGNIFICANT EH PREDICTORS (DIRECTION, WALD χ^2)

Context	BSFC (flow)	BSAP (anti-patterns)
Hibernate 5.0[r]	Throw Wrap (−, 5.8); # Invoked Methods (+, 6.2); Exc. Type Prevalence (−, 19.9)	Catch SLOC (+, 6.5); Sum AP Dummy Handler (+, 4.7)
Hibernate 5.1	Throw with New (+, 6.2); % Propagated (+, 15.8)	Avg AP Unhandled Exceptions (+, 12.1)
ZooKeeper 3.6[s]	Exc. Type Prevalence (−, 8.3)	none
ZooKeeper 3.7[s]	none	none
ZooKeeper 3.9[s]	% Handled with Return (+, 5.2)	none

[r] reproduction of the paper's Table 5 (our χ^2 equals the paper's to rounding). [s] under S1; the as-released ZooKeeper models drop BASE predictors (Obs. 3).

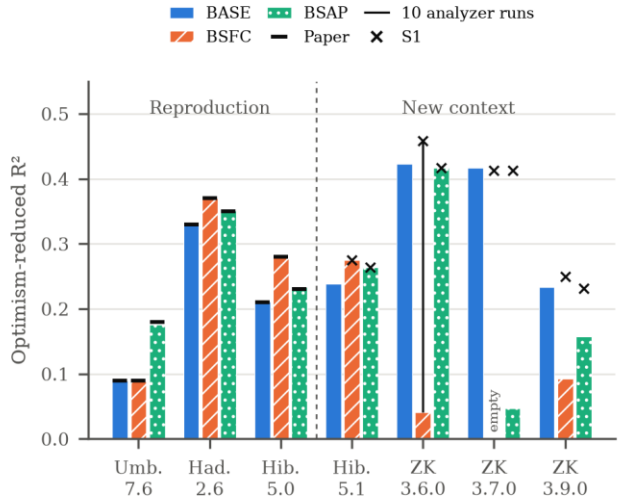


Fig. 1. Optimism-reduced R^2 of BASE, BSFC and BSAP. Left: reproduction on the authors' data, with the paper's Table 4 values (ticks). Right: our new releases, with the range over 10 analyzer runs (line) and the S1 sensitivity (×). ZooKeeper 3.7 BSFC is empty (no predictor survives).

Differences from the original, explained. Besides the systems themselves, three design differences separate the new contexts from the paper. (i) BASE uses 12 fallback product metrics instead of 39 Understand metrics (8 instead of 18 potential predictors on Hibernate); this should weaken BASE, yet BASE is higher on 5.1 (0.238 vs. 0.21), so we compare EH *increments* rather than absolute R^2. (ii) JTratch finds ~25% fewer possible exceptions without the authors' Javadoc database and dependency jars, which compresses flow-metric variation and can only make BSFC gains harder to detect. (iii) ZooKeeper releases have 161–178 modelled files and budgets of 10–11 predictors (Hibernate: 26–29), so few EH metrics survive the reduction steps. The ZooKeeper anomaly itself is explained by the model-construction code, not by the data (Obs. 3, Sec. IV-B).

C. Prediction (RQ3): reproduction contexts and new context

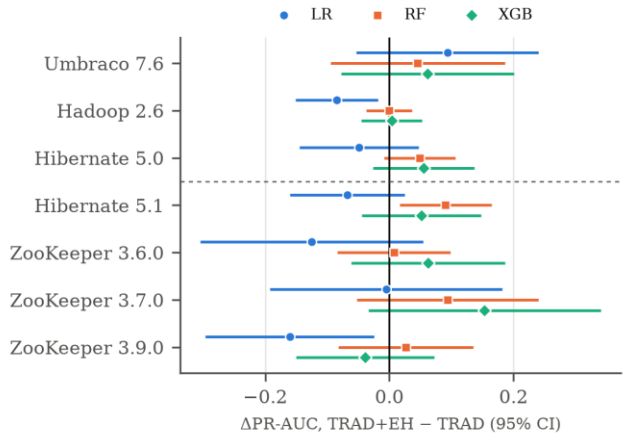


Fig. 2. Change in PR-AUC when EH metrics are added (TRAD+EH − TRAD), mean over 50 CV folds with unadjusted 95% corrected-t intervals. Above the dashed line: authors' data; below: our new releases. No interval survives Holm adjustment except one loss (LR, ZooKeeper 3.9, ROC-AUC).

Observation 4. EH metrics do not detectably improve prediction in any context (Fig. 2, Table VI). After Holm correction, 0 of 189 uplift tests show a gain; the single significant result is a **loss**: logistic regression with all EH features on ZooKeeper 3.9 (ROC-AUC −0.173, improved in 0 of 50 folds, p_Holm = 0.010). Point estimates differ by learner: tree models gain in 34 of 42 PR-AUC comparisons (largest: random forest on Hibernate 5.1, +0.091, raw p = 0.017, p_Holm = 0.16), whereas logistic regression loses in 16 of 21. This also corrects our earlier reading of the authors' data: the Umbraco ROC-AUC gains (raw p = 0.02) do not survive correction (p_Holm ≥ 0.17).

TABLE VI
PREDICTION AND SHAP RESULTS PER CONTEXT

Context	Prev.	Best PR-AUC TRAD	Best PR-AUC +EH	Holm-sig.	SHAP match
Reproduction contexts (authors' data)					
Umbraco 7.6	0.096	0.350	0.395	0/18	2/10
Hadoop 2.6	0.161	0.597	0.597	0/18	6/10
Hibernate 5.0	0.211	0.506	0.555	0/18	3/10
New context (our data)					
Hibernate 5.1	0.213	0.547	0.628	0/18	5/10
ZooKeeper 3.6	0.186	0.642	0.630	0/18	0/10
ZooKeeper 3.7	0.100	0.592	0.706	0/18	1/10
ZooKeeper 3.9	0.219	0.538	0.496	1/18[a]	5/10

Prev. = share of defective files (a random classifier's PR-AUC). Holm-sig. = PR-AUC and ROC-AUC uplift tests significant after Holm. SHAP match = top-10 EH features matching a significant BSFC/BSAP predictor. [a] a loss (LR, ROC-AUC).

D. Explanation (RQ4)

Observation 5. A traditional metric is the top SHAP feature in six of seven contexts (size, pre-release churn or pre-release bugs). Agreement with RQ1–RQ2 varies widely: 0–6 of the top-10 EH features match a significant predictor (Table VI). On the authors' data every match has the same sign as the regression coefficient; on Hibernate 5.1 one does not (`# Handled with Throw with New`). Two new-context exceptions are informative: on ZooKeeper 3.7 an anti-pattern (dummy handler) is the top feature overall and matches the only significant BSAP predictor, and on ZooKeeper 3.6 two of the top-5 features (`Avg Depth`, `Avg Max Depth`) are exactly the flow metrics that vary between analyzer runs (Sec. IV-B).

IV. Interpretation and Validity

A. What the evidence supports, and what it does not

Supported (strong). The paper's explanatory results are computationally correct but cannot be regenerated from the released package without our fix; the reproduction is exact.

Supported (moderate). One release later, Hibernate's EH metrics still add a small amount of explanatory power (+0.03 to +0.04 R^2). The evidence supports *EH metrics carry some extra information about defect-prone files*, not *a particular EH practice causes defects*: the significant EH predictors change between releases.

Weak, context-dependent. On ZooKeeper the gain is −0.006 to +0.035 once the construction artefact is removed; with 17–39 defective files and budgets of 10–11 the models have little power, so this is an absence of evidence, not evidence of absence.

RQ3: no detectable predictive gain. The intervals (e.g., Hibernate 5.1 RF [+0.017, +0.164] unadjusted) do not exclude moderate gains for tree models, so we claim *no gain detectable with seven contexts and Holm correction*, not *no gain*. The logistic-regression losses are expected: unlike the paper's models, it receives ~100 correlated EH features with no reduction step.

B. Strongest threat: flow-metric measurement on new data (check V1, run)

The most serious alternative explanation for any RQ1 result on new data is that it reflects how JTratch happened to run, not the code: the analyzer is not run-to-run deterministic and, without the authors' Javadoc database and dependency jars, finds ~25% fewer possible exceptions. Check V1 (Fig. 3) shows the following. (i) Only 6 of 107 metrics ever change, all flow metrics (`Avg Depth` in up to 71 of 161 files); all 45 anti-pattern and 5 throws metrics are identical in all 30 runs, so **RQ2 results do not depend on the analyzer run**. (ii) BASE and BSAP are identical in all 30 runs; BSFC is identical in all runs for ZooKeeper 3.7 and 3.9. (iii) For ZooKeeper 3.6, BSFC takes three values (0.041 ×6, 0.108 ×2, 0.458 ×2). The mechanism is not large metric changes: the perturbation moves the budget-based correlation cutoff (0.24 vs. 0.30), which decides whether `# Bugs (pre)` survives. S1 removes this dependence (0.458, identical to the runs in which it survives). We therefore report new-context RQ1 results with their run range, rely on S1 for the RQ1 comparison, and treat flow-metric SHAP rankings on ZooKeeper 3.6 (Obs. 5) as unstable.

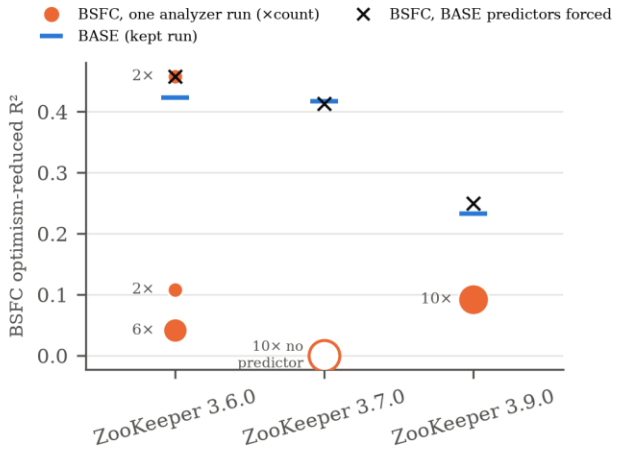


Fig. 3. Check V1: BSFC R^2 over 10 fresh analyzer runs per ZooKeeper release (dot size = number of runs with that value), against BASE and the S1 sensitivity. Only ZooKeeper 3.6 is affected, through the model-construction cutoff.

C. Second threat: defect-label precision (check V2, planned)

Key-based issue–commit linking misses unlinked fixes and can include tangled or partial changes [19]. The 131-link packet is ready; automatic pre-checks already flag 23 potential problems (mostly commits that mention several issue keys, one revert). Dev and Dhruv rate independently (three yes/no/unsure questions per link), then adjudicate; we report Cohen's κ and precision with a 95% Wilson interval. **Decision rule:** if precision is below 0.8 in a context, we refit its models without the invalid links and report both results. Other threats are documented: the new-context BASE uses 12 fallback metrics (an extension, not a faithful BASE), and the ML learners are deliberately untuned.

V. Remaining Work and Contributions

A. Scope reductions and plan to Milestone 4

Scope reductions (Table II): Hadoop 2.7, the C# new context, and the Javadoc/dependency-jar fidelity fix are dropped; new-context evidence is therefore Java-only and comes from two systems. Table VII gives the remaining plan, each item with a fallback.

TABLE VII
Plan to Milestone 4

Task	Owner	By	Fallback
Rate 131 links, adjudicate; κ, precision; refit if precision < 0.8 (V2)	Dev, Dhruv	Nov 16	Rate Hibernate 5.1 and ZooKeeper 3.6 first (80 links)
Analyzer variance on Hibernate 5.1 (10 runs × 35 min; resumable)	Dev	Nov 20	5 runs; report as range
Report bug D1 (and D7, S1) to the original authors	Dhruv	Nov 13	Document in the final report
Show variance range and S1 in the dashboard	Pulkit	Nov 24	Keep the existing caveat flags
Final report: RQ answers, threats, replication summary	All (lead Dhruv)	Nov 30	—

B. Completed contributions

TABLE VIII
MEMBER CONTRIBUTIONS (DETAIL IN CONTRIBUTIONS.MD)

Member	Completed (Milestones 2–3)	Main artifacts
Dev Patel	JTratch runs, aggregation re-implementation and validation; analyzer-variance check V1; ZooKeeper 3.7 collection; D7 discovery; rating packets; rater 1	`analyzer_variance.py`, `aggregate_jtratch.py`, `validation_packet.py`, `results/new/*/variance/`
Dhruv Patel	R reproduction and BSFC bug fix; sensitivity S1; D7 guard and verification; Holm correction; cross-context summary and figures; agreement script; rater 2; report lead	`run_original.R`, `context_summary.py`, `train_eval.py`, `validation_agreement.py`, `results/figures/`
Priyanka Gupta	Process/product metrics, Jira defect mining (all new contexts), collection tables, rating protocol	`defect_mining.py`, `process_metrics.py`, `docs/DATA.md`, `VALIDATION GUIDE.md`
Slesha Vora	RQ3/RQ4 protocol and runs on all seven contexts; SHAP alignment; notebook	`src/ml/`, `src/explain/`, `results/new/*/ml/`
Pulkit Chopra	Environment and docs; dashboard; statistics unit tests	`dashboard/`, `tests/`, `README.md`

REFERENCES

[1] G. B. de Pádua and W. Shang, "Studying the relationship between exception handling practices and post-release defects," in Proc. 15th Int. Conf. Mining Software Repositories (MSR), 2018, pp. 564–575, doi: 10.1145/3196398.3196435.

[2] G. B. de Pádua and W. Shang, "Revisiting exception handling practices with exception flow analysis," in Proc. IEEE 17th Int. Working Conf. Source Code Analysis and Manipulation (SCAM), 2017, pp. 11–20, doi: 10.1109/SCAM.2017.16.

[3] G. B. de Pádua and W. Shang, "Studying the prevalence of exception handling anti-patterns," in Proc. 25th Int. Conf. Program Comprehension (ICPC), 2017, pp. 328–331, doi: 10.1109/ICPC.2017.1.

[4] B. Cabral and P. Marques, "Exception handling: A field study in Java and .NET," in ECOOP 2007, LNCS 4609, 2007, pp. 151–175, doi: 10.1007/978-3-540-73589-2_8.

[5] D. Sena, R. Coelho, U. Kulesza, and R. Bonifácio, "Understanding the exception handling strategies of Java libraries: An empirical study," in Proc. 13th Int. Conf. Mining Software Repositories (MSR), 2016, pp. 212–222, doi: 10.1145/2901739.2901757.

[6] R. Coelho, L. Almeida, G. Gousios, A. van Deursen, and C. Treude, "Exception handling bug hazards in Android," Empirical Softw. Eng., vol. 22, no. 3, pp. 1264–1304, 2017, doi: 10.1007/s10664-016-9443-7.

[7] F. Khomh, M. Di Penta, Y.-G. Guéhéneuc, and G. Antoniol, "An exploratory study of the impact of antipatterns on class change- and fault-proneness," Empirical Softw. Eng., vol. 17, no. 3, pp. 243–275, 2012, doi: 10.1007/s10664-011-9171-y.

[8] S. E. S. Taba, F. Khomh, Y. Zou, A. E. Hassan, and M. Nagappan, "Predicting bugs using antipatterns," in Proc. IEEE Int. Conf. Software Maintenance (ICSM), 2013, pp. 270–279, doi: 10.1109/ICSM.2013.38.

[9] M. D'Ambros, M. Lanza, and R. Robbes, "An extensive comparison of bug prediction approaches," in Proc. 7th IEEE Working Conf. Mining Software Repositories (MSR), 2010, pp. 31–41, doi: 10.1109/MSR.2010.5463279.

[10] R. Moser, W. Pedrycz, and G. Succi, "A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction," in Proc. 30th Int. Conf. Software Engineering (ICSE), 2008, pp. 181–190, doi: 10.1145/1368088.1368114.

[11] N. Munaiah, S. Kroh, C. Cabrey, and M. Nagappan, "Curating GitHub for engineered software projects," Empirical Softw. Eng., vol. 22, no. 6, pp. 3219–3253, 2017, doi: 10.1007/s10664-017-9512-6.

[12] F. E. Harrell, Jr., Regression Modeling Strategies, 2nd ed. Cham, Switzerland: Springer, 2015, doi: 10.1007/978-3-319-19425-7.

[13] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 4765–4774.

[14] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[15] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[16] G. B. de Pádua and W. Shang, "Replication package: Studying the relationship between exception handling practices and post-release defects," 2018. [Online]. Available: https://guipadua.github.io/eh-model-defects2018/ (accessed Oct. 24, 2026).

[17] Group 8, "exception-handling-defects-replication," GitHub repository, 2026, tag milestone-3. [Online]. Available: https://github.com/DevPatel1704/exception-handling-defects-replication

[18] J. Sliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" in Proc. Int. Workshop Mining Software Repositories (MSR), 2005, pp. 1–5, doi: 10.1145/1083142.1083147.

[19] C. Bird, A. Bachmann, E. Aune, J. Duffy, A. Bernstein, V. Filkov, and P. Devanbu, "Fair and balanced? Bias in bug-fix datasets," in Proc. 7th Joint Meeting ESEC/FSE, 2009, pp. 121–130, doi: 10.1145/1595696.1595716.

[20] N. J. D. Nagelkerke, "A note on a general definition of the coefficient of determination," Biometrika, vol. 78, no. 3, pp. 691–692, 1991, doi: 10.1093/biomet/78.3.691.

[21] C. Nadeau and Y. Bengio, "Inference for the generalization error," Machine Learning, vol. 52, no. 3, pp. 239–281, 2003, doi: 10.1023/A:1024068626366.

[22] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian J. Statistics, vol. 6, no. 2, pp. 65–70, 1979.

[23] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.

[24] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.